Graph Algorithms

Part-1: Traversal and Topological Sort

Thanh-Sach LE

✉ ltsach@hcmut.edu.vn

Faculty of Computer Science and Engineering
Ho Chi Minh City University of Technology

Ho Chi Minh City on April 10, 2026

Outline

1. **Graph traversal**
   - Depth-first search
   - Breadth-first search
   - DFS vs BFS
2. **Topological sort**
   - DFS-based topological sort
   - Kahn's algorithm (queue-based)

ltsach@hcmut.edu.vn
Thanh-Sach LE

Assumed C++ Headers

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <algorithm>
#include <limits>
#include <tuple>
using namespace std;
```

- Code snippets focus on algorithm logic.
- Graphs are usually stored as adjacency lists.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Notation Used in This Deck

- n = number of vertices, m = number of edges.
- `adj[u]` = outgoing neighbors of vertex u.
- Weighted graph: `adj[u]` stores pairs `(v, w)`.
- Default representation: adjacency list (unless noted otherwise).

ltsach@hcmut.edu.vn
Thanh-Sach LE

Graph traversal

Outline
4 Graph traversal
Depth-first search
Breadth-first search
DFS vs BFS
Topological sort

ltsach@hcmut.edu.vn
Thanh-Sach LE

Why Graph Traversal?

- Traversal means systematically visiting reachable vertices/edges.
- Core use cases:
  - connectivity and components
  - search and reachability
  - cycle detection
  - shortest paths in unweighted graphs
  - topological sorting
- DFS and BFS are the foundation for most graph algorithms.

Outline
5 Graph traversal
Depth-first search
Breadth-first search
DFS vs BFS
Topological sort

ltsach@hcmut.edu.vn
Thanh-Sach LE

Common Traversal State

```
vector<bool> visited(n, false);
vector<int> parent(n, -1);
vector<int> order;
vector<int> dist(n, -1); // often used in BFS
```

- `visited` prevents infinite loops in cyclic graphs.
- `parent` reconstructs traversal tree or path.
- `order` records visitation sequence.
- `dist` stores shortest edge-count levels in BFS.

Outline
6 Graph traversal
Depth-first search
Breadth-first search
DFS vs BFS
Topological sort

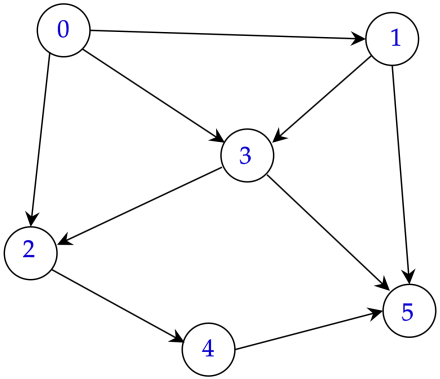
ltsach@hcmut.edu.vn
Thanh-Sach LE

What Is DFS?

- DFS explores as deeply as possible before backtracking.
- Stack behavior:
  - recursive DFS uses call stack
  - iterative DFS uses explicit `stack<int>`
- Useful for components, cycle analysis, topological sort, SCC.

ltsach@hcmut.edu.vn
Thanh-Sach LE

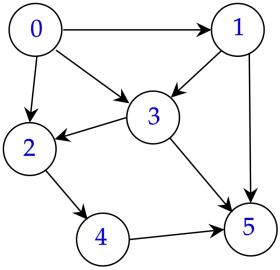
DFS Visualization: Initial Graph

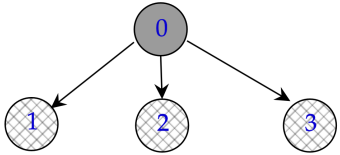


Start: 0. Visit unvisited neighbors in ascending order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 1





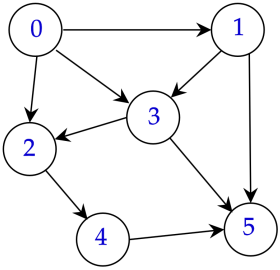
Visit unvisited neighbors in ascending order.

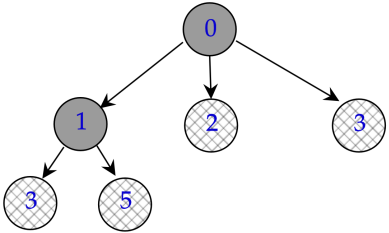
	0	1	2	3	4	5
visited:	true	false	false	false	false	false
parent:	−1	−1	−1	−1	−1	−1

order: 0

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 2



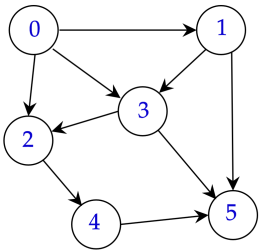


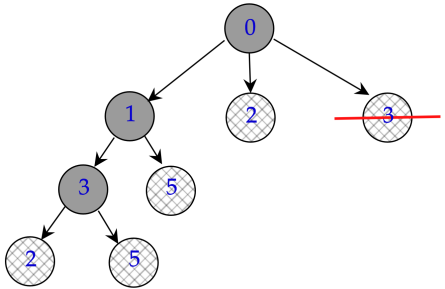
Visit unvisited neighbors in ascending order.

visited:	true	true	false	false	false	false
	0	1	2	3	4	5
parent:	−1	0	−1	−1	−1	−1
	0	1	2	3	4	5
order:	0	1				

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 3



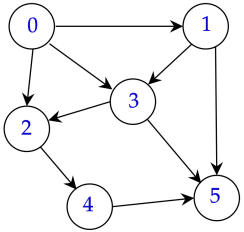


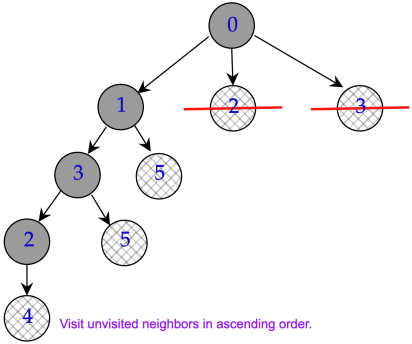
Visit unvisited neighbors in ascending order.

visited:	true	true	false	true	false	false
	0	1	2	3	4	5
parent:	−1	0	−1	1	−1	−1
	0	1	2	3	4	5
order:	0	1	3			

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 4

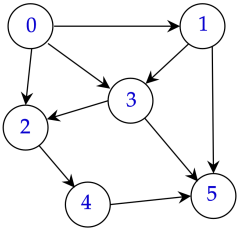


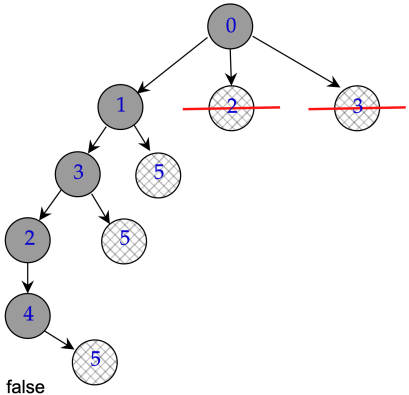


visited:	true	true	true	true	false	false
	0	1	2	3	4	5
parent:	−1	0	3	1	−1	−1
	0	1	2	3	4	5
order:	0	1	3	2		

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 5



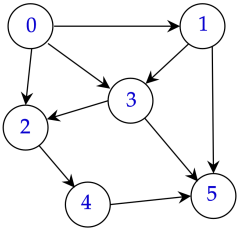


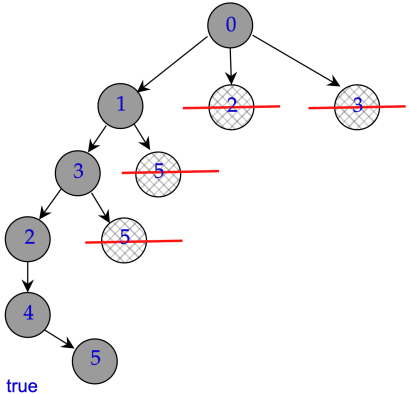
visited:	true	true	true	true	true	false
	0	1	2	3	4	5
parent:	−1	0	3	1	2	−1
	0	1	2	3	4	5
order:	0	1	3	2	4	

Visit unvisited neighbors in ascending order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Visualization: Step 6





visited:	true	true	true	true	true	true
	0	1	2	3	4	5
parent:	−1	0	3	1	2	4
	0	1	2	3	4	5
order:	0	1	3	2	4	5

Visit unvisited neighbors in ascending order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Recursive DFS (Adjacency List)

```
void dfs(int u, const vector<vector<int>>& adj,
         vector<bool>& visited, vector<int>& order) {
    visited[u] = true;
    order.push_back(u);

    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, adj, visited, order);
        }
    }
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Recursive DFS: Usage

```
vector<bool> visited(n, false);
vector<int> order;
dfs(0, adj, visited, order);
```

- Visits only vertices reachable from source 0.
- Add outer loop for disconnected graphs.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS with Parent Array

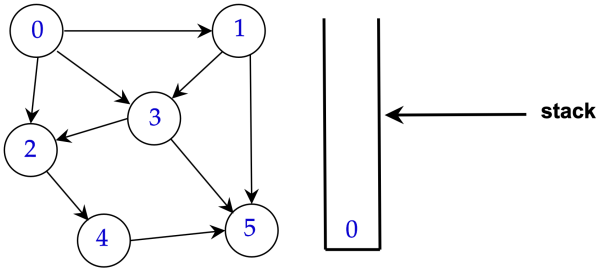
```
void dfs(int u, int p, const vector<vector<int>>& adj,
         vector<bool>& visited, vector<int>& parent) {
    visited[u] = true;
    parent[u] = p;

    for (int v : adj[u]) {
        if (!visited[v]) {
            dfs(v, u, adj, visited, parent);
        }
    }
}
```

- `parent` builds DFS tree.
- Useful for path and cycle analysis.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (0)

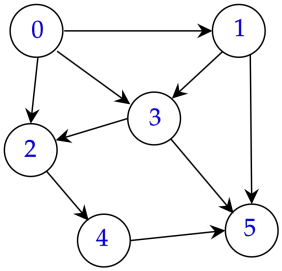


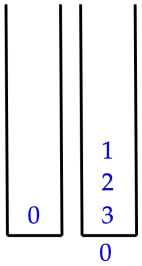
	0	1	2	3	4	5
visited:	false	false	false	false	false	false
parent:	−1	−1	−1	−1	−1	−1

order: _____

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (1)





Reverse pushing

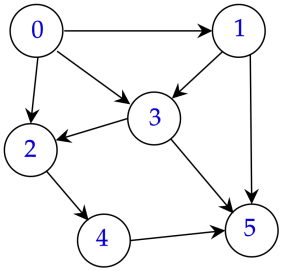
$\Rightarrow$ mimic recursive neighbor order.

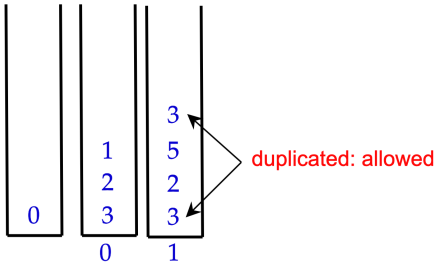
visited:	true	false	false	false	false	false
	0	1	2	3	4	5
parent:	−1	−1	−1	−1	−1	−1
	0	1	2	3	4	5
order:	0					

ltsach@hcmut.edu.vn

Thanh-Sach LE

Iterative DFS Stack Visualization (2)



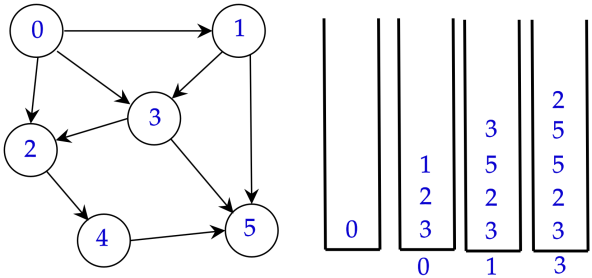


visited:	true	true	false	false	false	false
	0	1	2	3	4	5
parent:	−1	0	−1	−1	−1	−1
	0	1	2	3	4	5
order:	0	1				

Reverse pushing $\Rightarrow$ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (3)

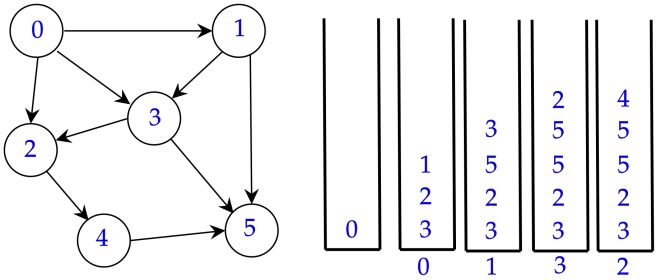


visited:	true	true	false	true	false	false
	0	1	2	3	4	5
parent:	−1	0	−1	1	−1	−1
	0	1	2	3	4	5
order:	0	1	3			

Reverse pushing $\Rightarrow$ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (4)

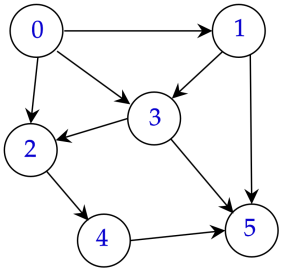


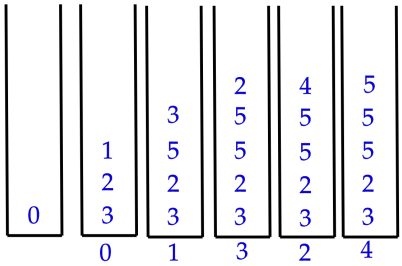
visited:	true	true	true	true	false	false
	0	1	2	3	4	5
parent:	−1	0	3	1	−1	−1
	0	1	2	3	4	5
order:	0	1	3	2		

Reverse pushing $\Rightarrow$ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (5)



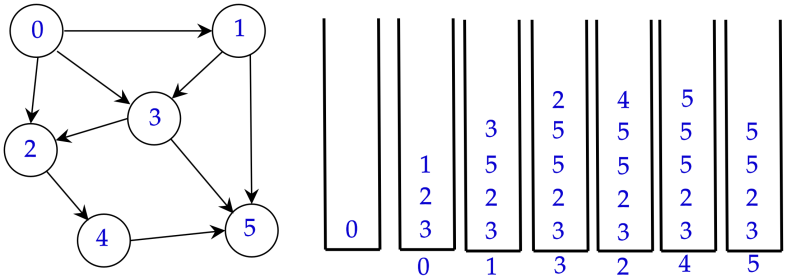


visited:	true	true	true	true	true	false
	0	1	2	3	4	5
parent:	−1	0	3	1	2	−1
	0	1	2	3	4	5
order:	0	1	3	2	4	

Reverse pushing $\Rightarrow$ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (6)

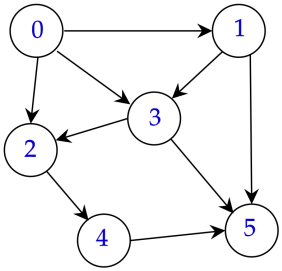


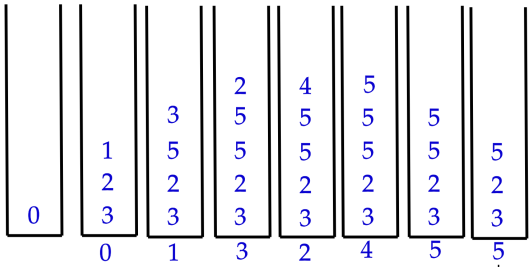
visited:	true	true	true	true	true	true
	0	1	2	3	4	5
parent:	−1	0	3	1	2	4
	0	1	2	3	4	5
order:	0	1	3	2	4	5

Reverse pushing $\Rightarrow$ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (7)





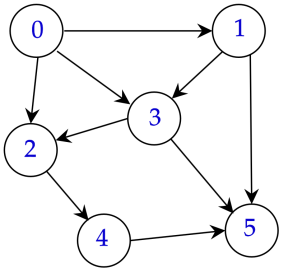
visited[5] = true ⇒ skip

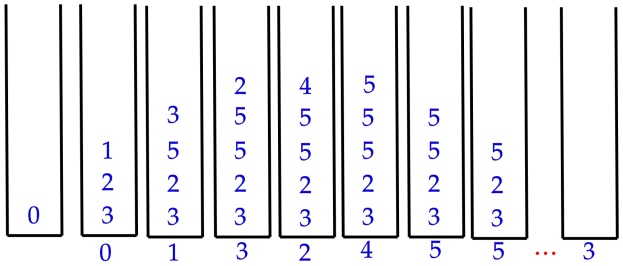
visited:	true	true	true	true	true	true
	0	1	2	3	4	5
parent:	−1	0	3	1	2	4
	0	1	2	3	4	5
order:	0	1	3	2	4	5

Reverse pushing ⇒ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS Stack Visualization (8)





visited:	true	true	true	true	true	true
	0	1	2	3	4	5
parent:	−1	0	3	1	2	4
	0	1	2	3	4	5
order:	0	1	3	2	4	5

visited[5] = true ⇒ skip
visited[2] = true ⇒ skip
visited[3] = true ⇒ skip
stack is empty ⇒ stop

Reverse pushing ⇒ mimic recursive neighbor order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS with Stack

```
vector<int> dfsIterative(int s, const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> order;
    stack<int> st;
    st.push(s);

    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (visited[u]) continue;
        visited[u] = true;
        order.push_back(u);
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Iterative DFS with Stack (continued)

```
        for (int i = (int)adj[u].size() - 1; i >= 0; --i) {
            int v = adj[u][i];
            if (!visited[v]) st.push(v);
        }
    }
    return order;
}
```

- Reverse pushing helps mimic recursive neighbor order.
- Some vertices may be pushed multiple times before first pop.

Outline
Graph traversal
28 Depth-first search
Breadth-first search
DFS vs BFS
Topological sort

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS for Disconnected Graphs

```
vector<int> dfsAll(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> order;

    for (int s = 0; s < n; ++s) {
        if (!visited[s]) {
            dfs(s, adj, visited, order);
        }
    }
    return order;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Complexity

Adjacency list

- each vertex visited once: $O(n)$
- each directed edge scanned once: $O(m)$
- undirected edge appears twice in list: still $O(n+m)$
- total: $O(n+m)$
- space: $O(n)$ plus recursion stack up to $O(n)$

Adjacency matrix

- for each visited vertex, scan a full row of n entries
- total: $O(n^2)$, space: $O(n)$

ltsach@hcmut.edu.vn
Thanh-Sach LE

Representation-specific DFS Notes

- Adjacency list is best for sparse graphs.
- Adjacency matrix is simple but scans many non-edges.
- Forward-star loop:

```
void dfs(int u) {
    visited[u] = 1;
    for (int e = head[u]; e != -1; e = nxt[e]) {
        int v = to[e];
        if (!visited[v]) dfs(v);
    }
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Common DFS Mistakes

- Forgetting `visited` array.
- Starting from one source on disconnected graph.
- Assuming DFS order is unique (depends on neighbor order).
- Stack overflow on very deep recursion.
- Confusing DFS-tree edges with all graph edges.

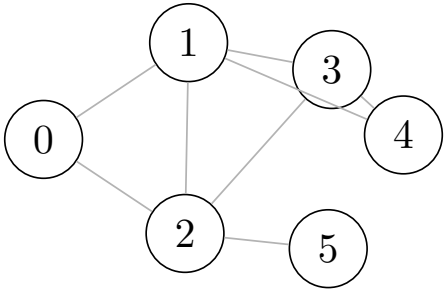
ltsach@hcmut.edu.vn
Thanh-Sach LE

What Is BFS?

- BFS explores level by level using a queue.
- In unweighted graphs, BFS gives shortest edge-count distance.
- Frontier expands uniformly by distance layers.

ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Intuition Graph and Levels

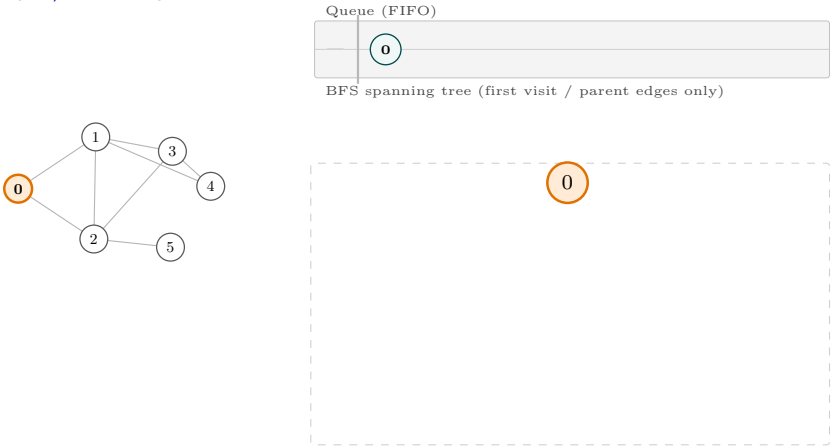


- Same graph as the step-by-step walk (not a tree: extra edges create cycles / alternate paths).
- Level 0: 0
- Level 1: 1, 2
- Level 2: 3, 4, 5

ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 1/7 — enqueue source 0

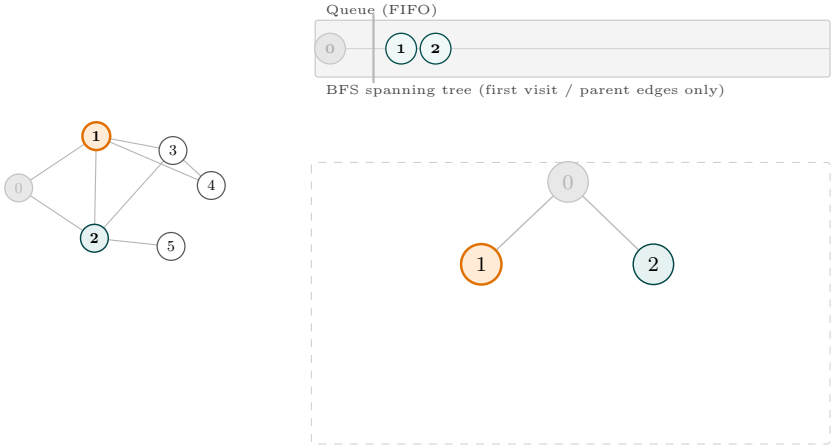


Left ≈40%*: full undirected graph (extra edges* 1–2, 2–3, 3–4 *vs. a tree). BFS still checks every neighbor and* **skips** *already-visited vertices. Right: FIFO queue + BFS spanning tree (first-discovery edges only). Orange = dequeuing front on* **both** *sides; teal = in queue; gray = done.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 2/7 — dequeue 0; enqueue 1, 2

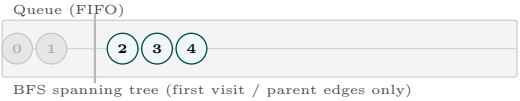


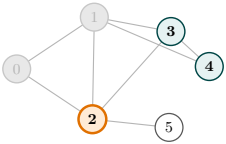
Outline
Graph traversal
Depth-first search
36 Breadth-first search
DFS vs BFS
Topological sort

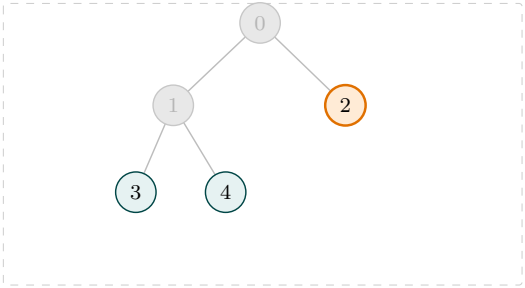
ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 3/7 — dequeue 1; enqueue 3, 4 (skip 2 visited)





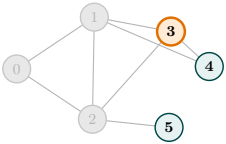


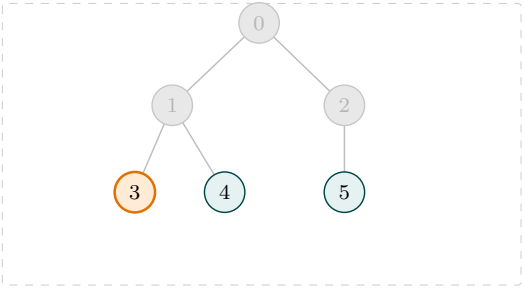
ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 4/7 — dequeue 2; enqueue 5 (skip 3 visited)





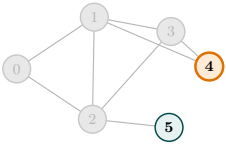


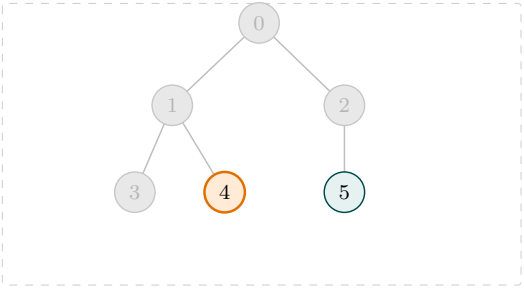
ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 5/7 — dequeue 3







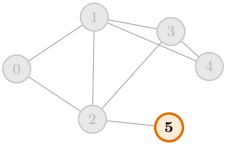
ltsach@hcmut.edu.vn

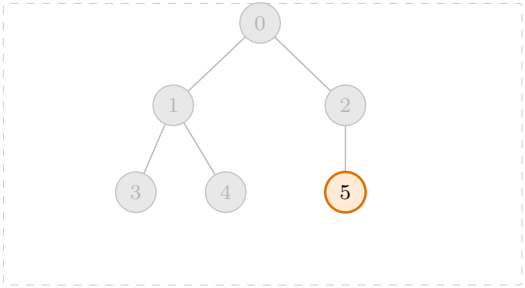
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 6/7 — dequeue 4





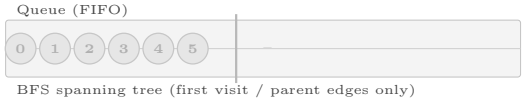


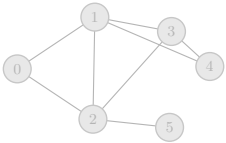
ltsach@hcmut.edu.vn

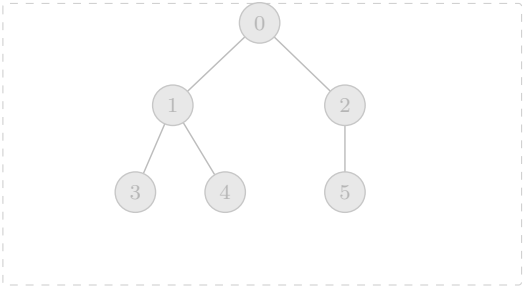
Thanh-Sach LE

BFS Step-by-step (Queue + tree)

Step 7/7 — dequeue 5; queue empty







BFS

visit order: $0, 1, 2, 3, 4, 5$ (same as dequeuing order here).

ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Code (Adjacency List)

```
vector<int> bfs(int s, const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> order;
    queue<int> q;

    visited[s] = true;
    q.push(s);
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Code (continued)

```
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);

        for (int v : adj[u]) {
            if (!visited[v]) {
                visited[v] = true; // mark when enqueuing
                q.push(v);
            }
        }
    }
    return order;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Distance and Parent

```
void bfsDistance(int s, const vector<vector<int>>& adj,
                 vector<int>& dist, vector<int>& parent) {
    int n = adj.size();
    dist.assign(n, -1);
    parent.assign(n, -1);

    queue<int> q;
    dist[s] = 0;
    q.push(s);
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Distance and Parent (continued)

```
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : adj[u]) {
            if (dist[v] == -1) {
                dist[v] = dist[u] + 1;
                parent[v] = u;
                q.push(v);
            }
        }
    }
}
```

- `dist[v]` = shortest number of edges from source to v.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Path Reconstruction from Parent

```
vector<int> buildPath(int t, const vector<int>& parent) {
    vector<int> path;
    for (int cur = t; cur != -1; cur = parent[cur]) {
        path.push_back(cur);
    }
    reverse(path.begin(), path.end());
    return path;
}
```

- Use only when target is reachable (`dist[t] != -1`).

ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS for Disconnected Graphs

```
vector<int> bfsAll(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> order;

    for (int s = 0; s < n; ++s) {
        if (!visited[s]) {
            queue<int> q;
            visited[s] = true;
            q.push(s);
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS for Disconnected Graphs (continued)

```
            while (!q.empty()) {
                int u = q.front(); q.pop();
                order.push_back(u);
                for (int v : adj[u]) {
                    if (!visited[v]) {
                        visited[v] = true;
                        q.push(v);
                    }
                }
            }
        }
    }
    return order;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

BFS Complexity

- Adjacency list: $O(n + m)$ time, $O(n)$ extra space.
- Adjacency matrix: $O(n^2)$ (scan full row each dequeue).
- Forward-star also gives $O(n + m)$ traversal.

Weighted graph note

BFS shortest path is valid only when all edges have equal weight. Use Dijkstra for nonnegative weighted edges.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Common BFS Mistakes

- Marking visited too late (on dequeue instead of enqueue).
- Assuming BFS solves weighted shortest path.
- Forgetting disconnected components.
- Not saving parent when path reconstruction is required.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS vs BFS Comparison

Feature	DFS	BFS
Data structure	stack / recursion	queue
Exploration style	deep first	level first
Shortest path (unweighted)	not guaranteed	guaranteed
Memory behavior	recursion/stack depth	queue frontier
Typical uses	components, cycle, topo	levels, unweighted shortest path

ltsach@hcmut.edu.vn
Thanh-Sach LE

Choosing DFS or BFS

- Need levels or shortest path in unweighted graph → BFS.
- Need deep exploration, components, cycle/topo reasoning → DFS.
- Need exhaustive search with backtracking → DFS.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Topological sort

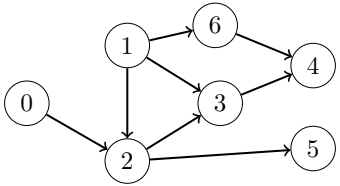
ltsach@hcmut.edu.vn
Thanh-Sach LE

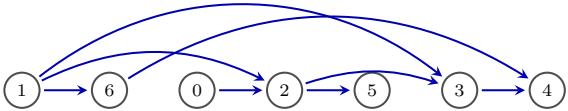
What Is Topological Sort?

- Input: directed acyclic graph (DAG).
- Output: order where each edge $u \to v$ has u before v.
- Defined only for DAG.
- Applications: prerequisites, build systems, scheduling.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DAG Example



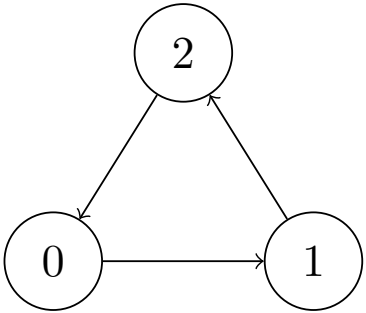


Same order $1, 6, 0, 2, 5, 3, 4$ on a line: each DAG edge drawn as a left→right arrow.

- Vertices 0 and 1 have indegree 0 (two sources). Example valid order: 1, 6, 0, 2, 5, 3, 4.
- **Quick check.** Write the candidate order on a line; it is a topological order iff *every* edge of the DAG points strictly left→right (no backward arrow). Often the fastest sanity check.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Why Cycles Make Topological Order Impossible



- In cycle $0 \to 1 \to 2 \to 0$, every vertex must appear before itself.
- Contradiction $\Rightarrow$ no topological order.

ltsach@hcmut.edu.vn
Thanh-Sach LE

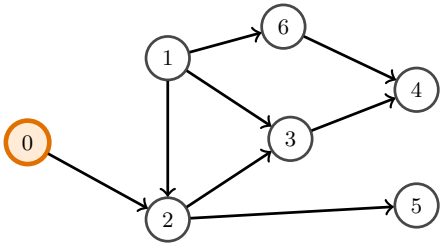
DFS-based Topological Sort Idea

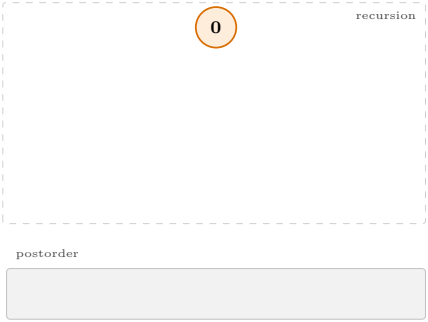
- Run DFS.
- Push vertex to order when DFS **finishes** that vertex.
- Reverse finishing order at the end.
- This is reverse postorder.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 1/19 — graph with 7 vertices; enter `topo(0)`



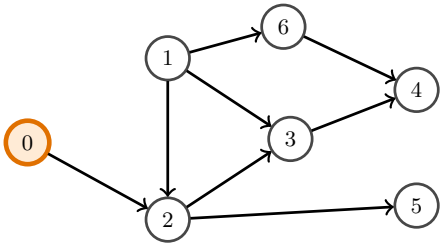


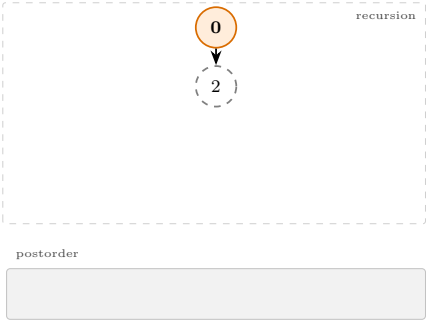
Left: graph; right: recursion panel + postorder strip. **Dashed***: all outgoing neighbors listed together (incl. next chosen);* **orange***: active after choice;* **muted gray***: node kept on tree after backtrack (off stack). Postorder strip grows left→right with each* `push_back`*.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 2/19 — from 0: neighbor [2] (all dashed)



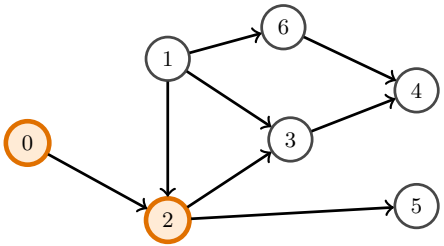


`adj[0]` = [2]: *only candidate* 2 *is still drawn dashed before the call commits.*

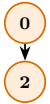
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 3/19 — choose 2 (highlight)



recursion



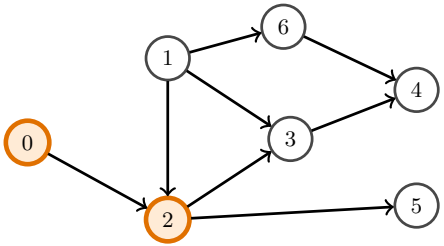
postorder

Solid edge + orange **2***: recursion enters* `topo(2)`*.*

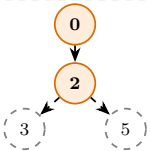
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 4/19 — from 2: neighbors [3,5] (both dashed)



recursion



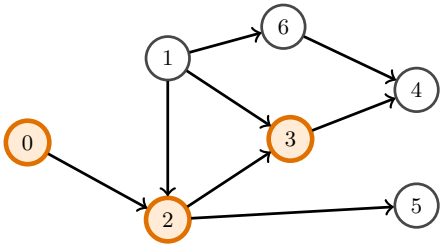
postorder

$adj[2] = [3, 5]$: *both appear dashed first (same style).*

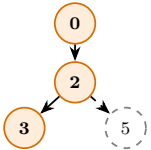
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 5/19 — choose 3



recursion



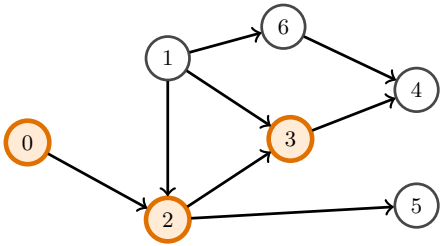
postorder

Smaller ID first: 3 *highlighted;* 5 *stays dashed (waiting).*

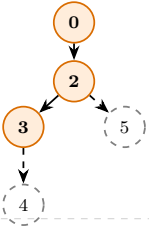
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 6/19 — from 3: neighbor [4] (dashed)



recursion



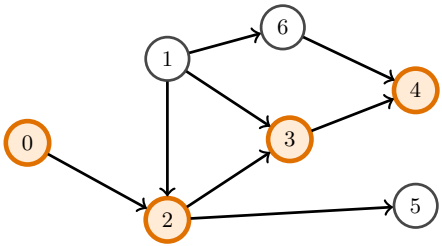
postorder

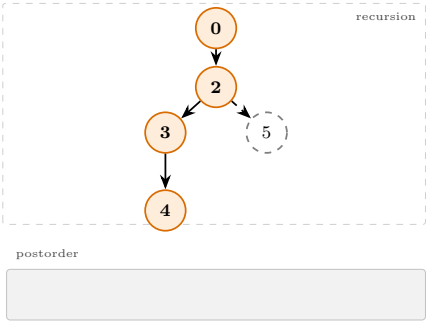
`adj[3]` = [4]: *sole child* 4 *dashed before descent.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 7/19 — choose 4



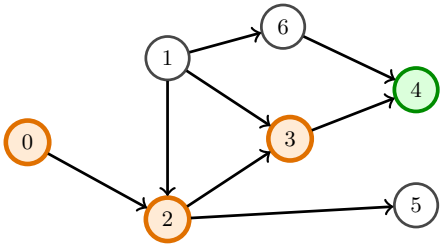


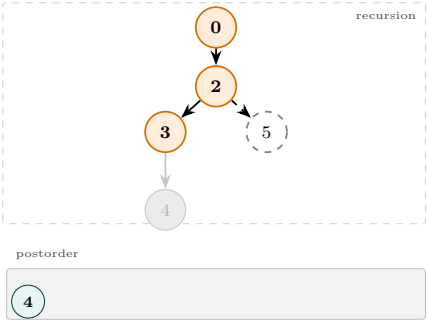
4 *becomes orange; deeper sink next.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 8/19 — backtrack from 4 (muted on tree)



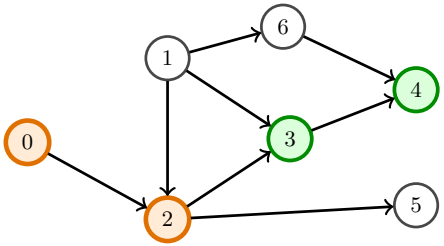


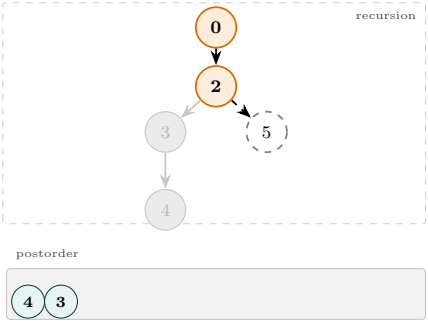
push_back(4). Node 4 stays in the picture but muted gray (frame returned).

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 9/19 — backtrack from 3



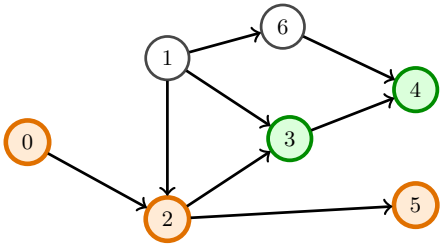


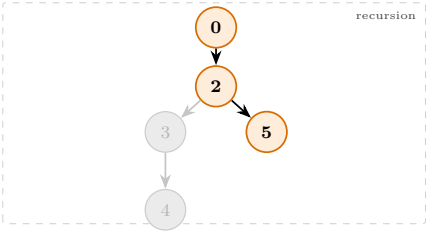
push_back(3); 3 *muted; next from* 2 *is* 5 *(still drawn dashed until chosen).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 10/19 — choose 5



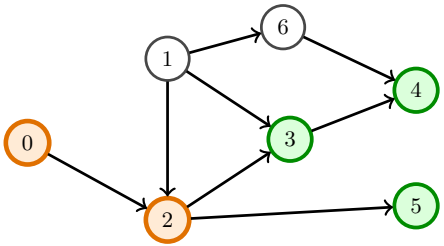


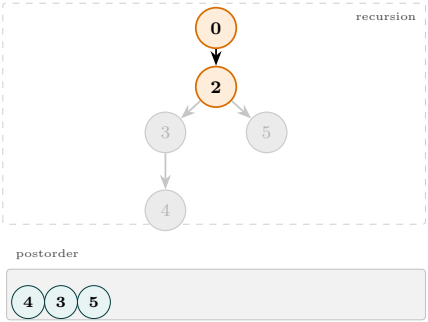
5 *switches from dashed to orange (highlight only after choice).*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 11/19 — backtrack from 5



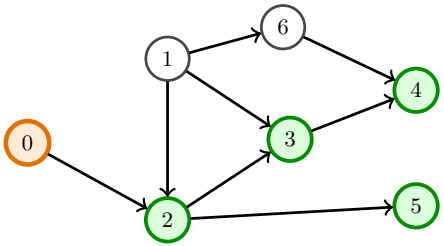


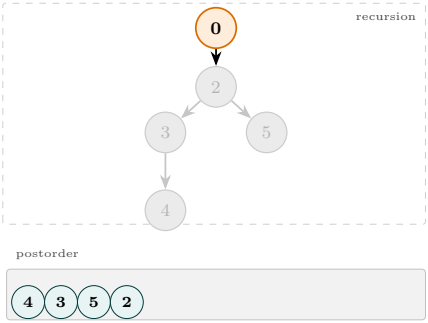
push_back(5); 5 *muted on tree.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 12/19 — backtrack from 2



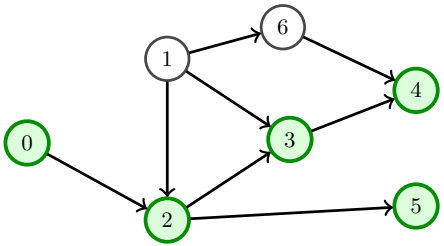


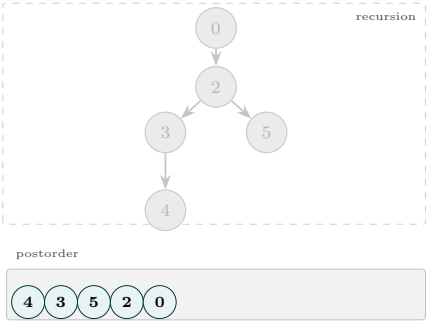
`push_back(2)`; *subtree under* 0 *collapsed visually but nodes remain as gray ghosts.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 13/19 — backtrack from 0



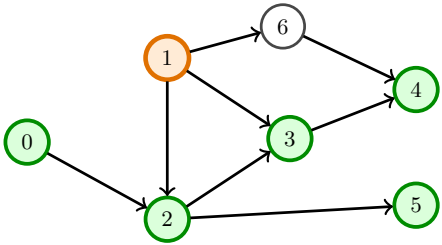


push_back(0). Component from source 0 *finished in postorder strip.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 14/19 — enter `topo(1)`



recursion

· · · 1

postorder

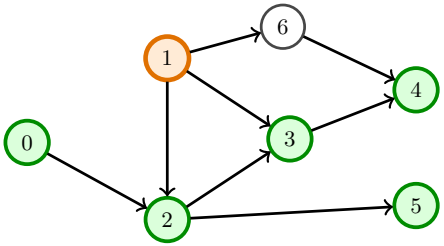
4 3 5 2 0

· · · *stands for the muted* 0*-branch already unwound.*

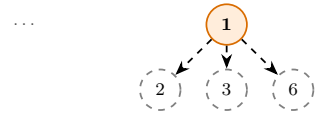
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 15/19 — from 1: neighbors [2,3,6] (all dashed)



recursion



postorder

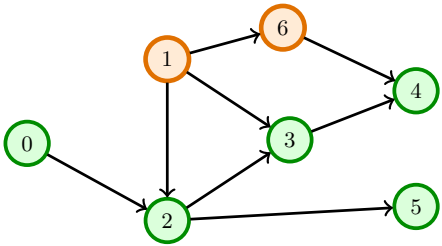
4 3 5 2 0

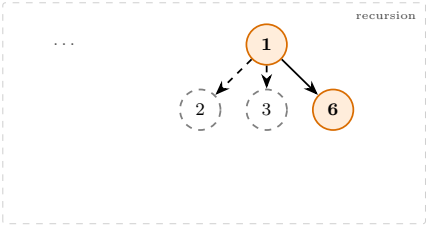
`adj[1]` = [2, 3, 6] *sorted; every candidate dashed together before any recursion.*

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 16/19 — skip 2, 3; choose 6



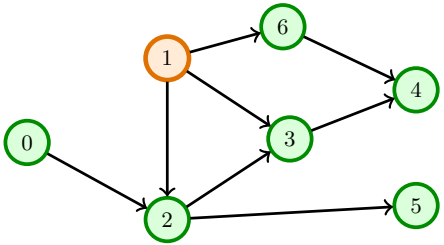


Visited neighbors stay dashed here (skip); only unvisited 6 *turns orange.*

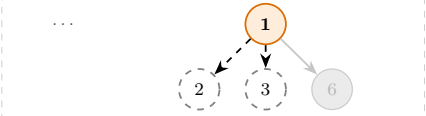
ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 17/19 — backtrack from 6 (`adj[6]=[4]` already done)



recursion



postorder

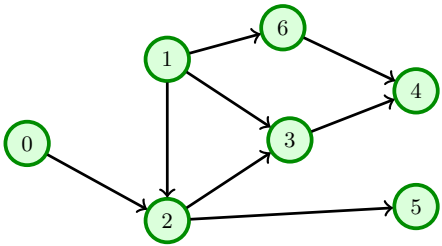


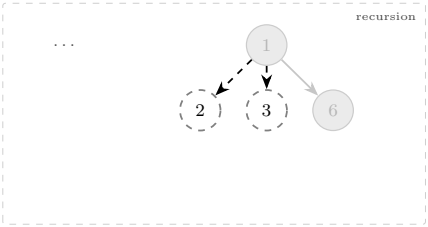
`push_back(6)`; 6 *muted; no new recursion into* 4.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 18/19 — backtrack from 1



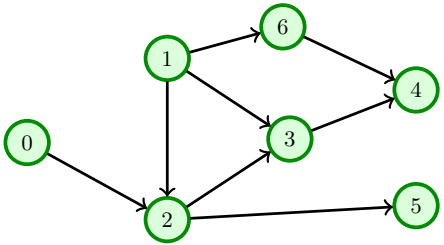


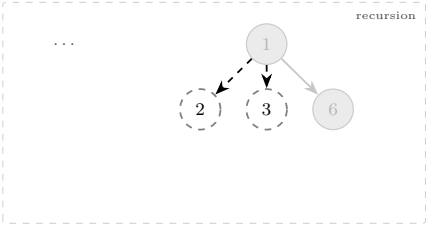
push_back(1): full postorder vector before reverse.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo Step-by-step (Concept)

Step 19/19 — reverse finishing list





postorder



Postorder (strip): $[4, 3, 5, 2, 0, 6, 1]$

$\Rightarrow$ **reverse** $\Rightarrow$ $[1, 6, 0, 2, 5, 3, 4]$

One valid topological order for this DAG.

Reverse yields a linear extension respecting all edges.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS-based Topological Sort Code

```
void topoDfs(int u, const vector<vector<int>>& adj,
             vector<bool>& visited, vector<int>& order) {
    visited[u] = true;
    for (int v : adj[u]) {
        if (!visited[v]) topoDfs(v, adj, visited, order);
    }
    order.push_back(u);
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS-based Topological Sort (Driver)

```
vector<int> topoSortDfs(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<bool> visited(n, false);
    vector<int> order;

    for (int u = 0; u < n; ++u) {
        if (!visited[u]) topoDfs(u, adj, visited, order);
    }
    reverse(order.begin(), order.end());
    return order;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Cycle Detection Requirement in DFS Topo

- A boolean `visited` alone is not enough to reject cycles.
- Use colors:
  - 0 = unvisited
  - 1 = visiting (in recursion stack)
  - 2 = finished
- Edge to color-1 vertex means back-edge $\Rightarrow$ cycle.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo with Cycle Detection

```
bool dfsCycle(int u, const vector<vector<int>>& adj,
              vector<int>& color, vector<int>& order) {
    color[u] = 1;
    for (int v : adj[u]) {
        if (color[v] == 1) return false;
        if (color[v] == 0) {
            if (!dfsCycle(v, adj, color, order)) return false;
        }
    }
    color[u] = 2;
    order.push_back(u);
    return true;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Topo Driver with Cycle Detection

```cpp
vector<int> topoSortDfsSafe(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> color(n, 0), order;
    for (int u = 0; u < n; ++u) {
        if (color[u] == 0) {
            if (!dfsCycle(u, adj, color, order)) return {};
        }
    }
    reverse(order.begin(), order.end());
    return order;
}
```


Outline
Graph traversal
Topological sort
81 DFS-based topological sort
Kahn's algorithm (queue-based)

ltsach@hcmut.edu.vn
Thanh-Sach LE

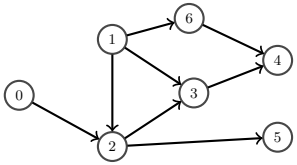
Kahn's Algorithm Definition

- Compute indegree of all vertices.
- Push all vertices with indegree 0 to queue.
- Repeatedly remove queue front and reduce indegree of neighbors.
- If processed count $< n$, cycle exists.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 1/9 — initialize indegrees & seed queue



queue: 0 1

Compute indegrees from edges. Enqueue every vertex with indeg $= 0$ (scan $0 \ldots n-1 \Rightarrow$ queue `[0,1]` here).

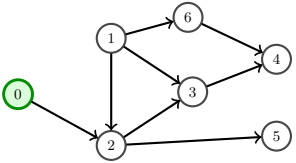
vertex	indeg
0	0
1	0
2	2
3	2
4	2
5	1
6	1

DAG (same graph as DFS topo) | FIFO queue strip + indegree table.

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 2/9 — dequeue 0; relax 0 → 2





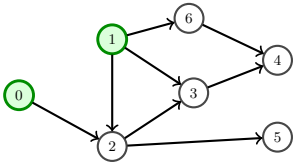
pop ⇒ 0. Relax 0 → 2: indeg[2]: 2 → 1.

vertex	indeg	
0	0	done
1	0	
2	2 $\xrightarrow{-1}$ **1**	
3	2	
4	2	
5	1	
6	1	

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 3/9 — dequeue 1; *relax* $1 \to 2, 3, 6$



0 1 **2** **6**

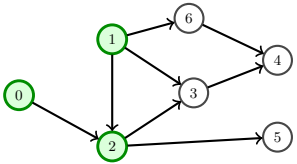
pop $\Rightarrow$ 1. Relax $1 \to 2, 3, 6$: indeg[2]: $1 \to 0$ (push 2); indeg[3]: $2 \to 1$; indeg[6]: $1 \to 0$ (push 6).
Queue becomes `[2,6]`.

vertex	indeg	
0	0	done
1	0	done
2	$2 \xrightarrow{-1} 1 \xrightarrow{-1} \mathbf{0}$	push
3	$2 \xrightarrow{-1} \mathbf{1}$	
4	2	
5	1	
6	$1 \xrightarrow{-1} \mathbf{0}$	push

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 4/9 — dequeue 2; relax 2 → 3, 5



0 1 2 **6** **3** **5**

pop ⇒ 2. Relax 2→3, 5: indeg[3]: 1→0 (push 3); indeg[5]: 1→0 (push 5). Queue `[6,3,5]`.

vertex	indeg	
0	0	done
1	0	done
2	0	done
3	2 $\xrightarrow{-1}$ 1 $\xrightarrow{-1}$ **0**	push
4	2	
5	1 $\xrightarrow{-1}$ **0**	push
6	0	

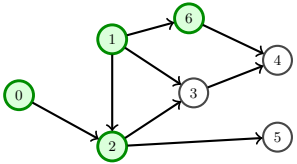
ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 5/9 — dequeue 6; relax 6 → 4



pop ⇒ 6. Relax 6→4: indeg[4]: 2→1. Queue [3,5].

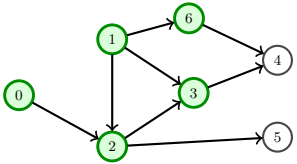


vertex	indeg	
0	0	done
1	0	done
2	0	done
3	0	
4	2 $\xrightarrow{-1}$ **1**	
5	0	
6	0	done

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 6/9 — dequeue 3; relax 3 → 4





pop ⇒ 3. Relax $3 \to 4$: indeg[4]: $1 \to 0$ (push 4).
Queue `[5,4]`.

vertex	indeg	
0	0	done
1	0	done
2	0	done
3	0	done
4	$2 \xrightarrow{-1} 1 \xrightarrow{-1} \mathbf{0}$	push
5	0	
6	0	done

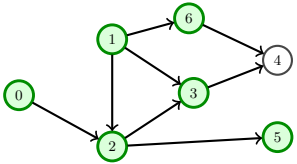
ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 7/9 — dequeue 5



pop $\Rightarrow$ 5. No outgoing edges from 5. Queue [4].

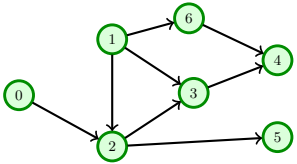


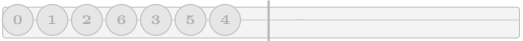
vertex	indeg	
0	0	done
1	0	done
2	0	done
3	0	done
4	0	
5	0	done
6	0	done

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 8/9 — dequeue 4; queue empty





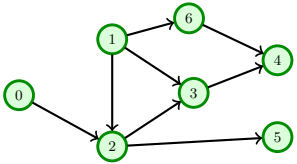
pop ⇒ 4. Queue empty; all vertices processed ⇒ DAG OK.

vertex	indeg	
0	0	done
1	0	done
2	0	done
3	0	done
4	0	done
5	0	done
6	0	done

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn Step-by-step

Step 9/9 — topological order (dequeuing order)



0 1 2 6 3 5 4

Topological order (dequeuing order):
0, 1, 2, 6, 3, 5, 4.
(Same DAG as the DFS topo walk; Kahn's dequeuing order need not match DFS postorder ∘ reverse.)

vertex	indeg
0	0
1	0
2	0
3	0
4	0
5	0
6	0

ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn's Algorithm Code

```
vector<int> topoSortKahn(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<int> indeg(n, 0);
    for (int u = 0; u < n; ++u)
        for (int v : adj[u]) indeg[v]++;

    queue<int> q;
    for (int u = 0; u < n; ++u)
        if (indeg[u] == 0) q.push(u);
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Kahn's Algorithm Code (continued)

```
    vector<int> order;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
        for (int v : adj[u]) {
            indeg[v]--;
            if (indeg[v] == 0) q.push(v);
        }
    }
    if ((int)order.size() != n) return {}; // cycle
    return order;
}
```


ltsach@hcmut.edu.vn
Thanh-Sach LE

Topological Sort Complexity

- Adjacency list:
  - DFS topo: $O(n + m)$
  - Kahn: $O(n + m)$
- Adjacency matrix:
  - often $O(n^2)$ due to full row/column scans
- Space: $O(n)$ extra.

ltsach@hcmut.edu.vn
Thanh-Sach LE

DFS Topo vs Kahn

DFS-based	Kahn
postorder + reverse	indegree + queue
needs color for cycle detection	cycle by processed count
fits DFS reasoning	fits scheduling simulation

ltsach@hcmut.edu.vn
Thanh-Sach LE

Common Topological Sort Mistakes

- Applying topo sort to undirected graph.
- Forgetting DAG requirement.
- Using DFS visited-only without cycle detection.
- Wrong indegree maintenance in Kahn.
- Assuming topological order is unique.

ltsach@hcmut.edu.vn
Thanh-Sach LE